

이슈 전수 정리 & 완성까지의 로드맵

노션 · 슬랙 · 지라를 하나로 잇는 협업 SaaS —
지금까지 처리한 **KAN-1 ~ KAN-22**의 회고와, 목표 제품에 도달하기 위한 **신규 티켓 29건**의 계획.

기준일	2026년 7월 25일
저장소	github.com/DankangE/DankangNote (main + 진행 브랜치 3)
이슈 트래커	ekff99.atlassian.net — 프로젝트 KAN
작성자	DanKang
문서 성격	진행 회고 + 백로그 계획 (기획서 v1.0의 후속 문서)

22

등록된 이슈

17

머지된 PR

6

DB 마이그레이션

29

계획 티켓

14일간 22개 이슈 중 16개를 완료했고 2개가 리뷰 중이다. 협업 SaaS의 세 기둥(문서·채팅·보드)이 모두 MVP 수준으로 동작하고, 멀티테넌시·인증·웹훅 동기화 같은 **SaaS 기반 골격**은 이미 프로덕션급 설계로 갖춰졌다. 반대로 **테스트·CI·배포**는 아직 0이며, 각 기능은 "1개 방·1개 보드·평면 문서" 수준의 MVP에 머물러 있다.

16 / 22

완료 이슈

Jira 샘플 3건 제외하면 16/19 · 리뷰 중
2건이 머지되면 18

17 머지 · 2 오픈

Pull Request

전 PR에 멀티 에이전트 자체 리뷰 → 확정
발견만 반영

61

main 커밋

기능 커밋 + [Fix] 리뷰 반영 커밋 쌍
구조

3,671 LOC

앱 코드 (71파일)

생성 코드 제외 · `src/` 기준

8

Prisma 모델

6회 마이그레이션으로 점진 확장

0

자동 테스트 · CI

가장 큰 구조적 공백 — Phase 6 최우선

이슈 상태 분포



목표 제품 대비 완성도

기획서의 4대 기능(F1~F4)과 운영 기반에 대해, 최종 목표(슬랙+노션 수준)를 100으로 둔 **주관적 추정치**다. 수치 자체보다 **어디가 비어 있는지를** 읽기 위한 그림이다.

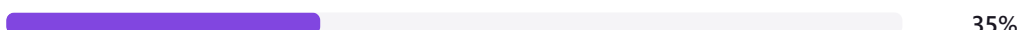
F1 워크스페이스·멤버·권한

인증/조직/역할



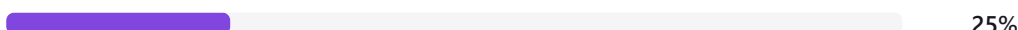
F2 문서 (노션형)

노트·에디터



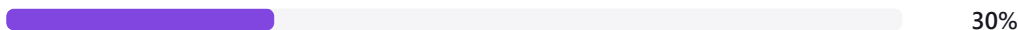
F3 채팅 (슬랙형)

실시간 메시징



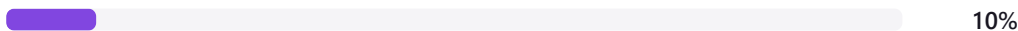
F4 보드 (지라형)

칸반



운영 기반

테스트·CI·배포·관측



이 리포트의 결론 세 줄

- 기반은 이미 강하다. 멀티테넌시 격리·웹훅 순서 역전 가드·세션 기반 authz는 실무 SaaS에서도 그대로 통하는 설계다.
- 제품은 아직 데모다. 채널 1개·보드 1개·평면 문서 — 슬랙/노션이라 부르려면 채널·스레드·문서 트리가 필요하다.
- 다음 스프린트는 기능이 아니라 정리다. 겹친 UI PR 2건 정리 → 테스트·CI·배포 → 그 다음에 기능 확장.

Phase 0 — 기반 세팅 07-11

Next.js 16(App Router)+TypeScript strict 초기화, 로컬 Postgres(docker-compose), Prisma 7 도입. 작업 규칙(`.claude/rules/`)과 "추측 금지 · 공식 문서 확인" 원칙을 이때 못 박았고, 이후 모든 스택 도입에 그대로 적용됐다.

Phase 1 — 노트 CRUD API와 멀티테넌시 골격 07-11 ~ 07-17

도메인 첫 수직 슬라이스. 모든 쿼리를 `orgId` 로 스코프하고 update/delete를 `*Many` 로 처리해 교차 테넌트 접근을 쿼리 수준에서 차단했다. PR #1 자체 리뷰에서 나온 8건을 KAN-5·6·7로 분리해 순차 처리 — "발견은 티켓으로 분리한다"는 운영 방식이 여기서 자리 잡았다.

KAN-4 CRUD API KAN-5 빌드 복구 KAN-6 에러 계약 KAN-7 리뷰 cleanup

Phase 2 — 첫 화면과 낙관적 UI 07-18

노트 목록·생성·편집·삭제 UI. 이어서 React 19 `useOptimistic` 으로 ghost card(삭제됐는데 남는 카드)와 저장 후 옛 값 깜빡임을 함께 해소했다.

KAN-8 노트 UI KAN-9 낙관적 업데이트

Phase 3 — 인증·멀티테넌시 실연동 07-18 ~ 07-23

Clerk 도입으로 임시 `org_dev` 를 실제 Organization으로 교체. 이어 웹훅으로 User/Organization/Membership을 DB에 미러링해 관계형 기능의 토대를 만들고, 첫 소비자로 멤버 목록 화면을 붙였다. 역등 upsert + 멤버십 이벤트 단일 트랜잭션으로 배달 순서와 무관한 FK 안전을 확보했다.

KAN-10 Clerk 뼈대 KAN-11 웹훅 동기화 KAN-13 멤버 목록

Phase 4 — 데이터 모델 성숙 + 3대 기능 완주 07-23 ~ 07-24

가장 밀도 높은 구간. 미리 무결성(순서 역전 가드), Note의 FK 소급·작성자 귀속을 마친 뒤 채팅·에디터·보드·권한을 연달아 올렸다. 이 시점에 기획서의 3대 핵심 기능이 모두 화면에서 동작하게 됐다.

KAN-12 순서 역전 가드 KAN-14 FK·작성자 KAN-15 실시간 채팅 KAN-16 Tiptap KAN-17 칸반 보드 KAN-18 역할 권한

Phase 5 — UI 스택 전환과 디자인 시스템 07-25

StyleX가 babel을 요구해 next/font·Turbopack과 충돌하는 것이 확인되어, UI 스택을 Astryx/StyleX → shadcn/ui + Tailwind v4로 전환(20파일 이관, 로직 무변경). 이어서 바이올렛 브랜드 토큰과 슬랙형 앱 셀까지 진행했다. 이 단계만 아직 main에 안착하지 못했다(5장 참조).

KAN-20 스택 전환 KAN-21 디자인·앱 셀 KAN-22 트렌디 폴리시

Phase 6~10 — 여기서부터가 이 문서의 계획 다음

정리·기반 → 슬랙형 채팅 심화 → 노선형 문서 심화 → 플랫폼 통합 → 운영 품질. 6장부터 상세.

키	제목	PR	실제 상태	남긴 것 · 핵심 결정
KAN-1	협업 SaaS 프로젝트	—	샘플	Jira 온보딩 시 자동 생성된 잔재. 정리 대상.
KAN-2	작업 2	—	샘플	동일 — 상태가 "진행 중"이라 보드 통계를 오염시킨다.
KAN-3	하위 작업 2.1	—	샘플	동일.
KAN-4	노트 CRUD API	#1	완료	수직 슬라이스 구조(<code>features/</code> + <code>server/</code>) 확립. 전 쿼리 <code>orgId</code> 스코프, <code>update/delete</code> 는 <code>updateMany / deleteMany</code> 로 교차 테넌트 차단. 모든 외부 입력 <code>zod</code> 검증.
KAN-5	fresh clone 빌드 실패 수정	#2	완료	생성 Prisma 클라이언트가 <code>gitignore</code> 대상이라 새 클론에서 빌드 불가 → <code>postinstall: prisma generate</code> .
KAN-6	Server Action 에러 계약 + 원자성	#3	완료	DB 예외가 <code>ActionResult</code> 유니언을 뚫고 나가지 않도록 <code>guarded</code> 래퍼. <code>updateNote</code> 재조회 경합 제거.
KAN-7	PR #1 리뷰 cleanup 일괄	#9	완료	<code>zod</code> 검증 일관성, 에러 추출 헬퍼 추출, 로컬 설정 파일 정리 등 5건.
KAN-8	노트 UI — 목록·생성·편집·삭제	#4	완료	첫 프론트엔드. 서버 컴포넌트 조회 + Server Action 변이 패턴 정착.
KAN-9	노트 낙관적 업데이트	#7	완료	<code>useOptimistic</code> 도입. 실패 복원이 사용자의 새 입력을 덮어쓰지 않게 하는 재조정 경로까지 처리.
KAN-10	Clerk 인증 뼈대	#5	완료	Organization = 워크스페이스 매핑 확정. <code>requireOrgId()</code> 가 실제 세션 <code>orgId</code> 반환.
KAN-11	Clerk webhook → DB 동기화	#6	완료	User-Organization-Membership 미러. Svix 서명 검증, 역등 <code>upsert</code> , 멤버십 이벤트는 <code>org+user+membership</code> 을 한 트랜잭션 에 <code>upsert</code> 해 배달 순서와 무관하게 FK 안전.
KAN-12	웹훅 미러 순서 역전 가드	#12	완료	<code>at-least-once</code> ·무순서 배달 대응. <code>ClerkTombstone</code> (삭제 영구 기록) + <code>clerkUpdatedAt</code> 이중 가드로 "삭제된 유저가 지연 이벤트로 부활"하는 유령 행을 차단. Jira 상태는 아직 "진행 중" — 정정 대상.
KAN-13	워크스페이스 멤버 목록 UI	#8	완료	미러 데이터의 첫 소비자. 미러 <code>role</code> 은 표시용 이며 권한 판단에 쓰지 않는다는 원칙 확립.
KAN-14	Note→Organization FK + 작성자 귀속	#10	완료	FK 소급 + 생성 트랜잭션의 스켈레톤 upsert 로 웹훅 지연과의 경합 해소. 조직 삭제 시 노트는 Cascade 파기 로 테넌트 수명 정책 확정.
KAN-15	실시간 채팅 MVP	#11	완료	Pusher private 채널(<code>private-org-<orgId></code>). 인증 라우트에서 요청 채널과 세션 <code>orgId</code> 일치 검증. 워크스페이스당 방 1개 스코프.
KAN-16	Tiptap 리치 텍스트 에디터	#13	완료	<code>content</code> 는 String 유지 + Tiptap doc JSON 직렬화 (마이그레이션 0). <code>zod</code> 노드/마크 화이트리스트 + <code>attrs strip</code> 으로 저장형 XSS 차단, 뷰는 <code>read-only</code> 렌더러.
KAN-17	칸반 보드 MVP	#14	완료	<code>BoardColumn</code> · <code>BoardCard</code> + <code>dnd-kit</code> 멀티 컨테이너 DnD. 이동 시 대상 컬럼 <code>position 0..n</code> 재작성, 실패 시 타깃 함수형 롤백 (전체 스냅샷 복원은 동시 변이를 덮어써서 기각).

키	제목	PR	실제 상태	남긴 것 · 핵심 결정
KAN-18	역할 기반 권한 강제	#15	완료	노트 수정·삭제 = 작성자 본인 또는 <code>org:admin</code> . 역할은 Clerk 세션 클레임으로만 판단하고, 소유권 조건을 <code>where</code> 절에 원자적으로 반영. UI 숨김은 편의일 뿐 강제는 서버.
KAN-19	ChatMessage 테넌트 수명 정렬	—	미착수	유일한 열린 보안 항목. ChatMessage는 설계상 FK가 없어 조직 삭제 <code>cascade</code> 가 닿지 않고, <code>createMessage</code> 에 <code>tombstone</code> 체크도 없다 → 고아 메시지가 영구 잔존 가능.
KAN-20	UI 스타크 전환 (shadcn/ui + Tailwind v4)	#16	완료	StyleX가 babel 필수 → next/font-Turbopack과 충돌함을 확인하고 스타크 전환. 20파일 이관·Astryx 잔재 0·로직 무변경. Jira 제목은 아직 옛 범위(StyleX 빌드 설정) — 정정 대상.
KAN-21	디자인 시스템 + 슬랙형 앱 셀	#17 머지 #19 오픈	리뷰 중	1단계(바이올렛 토큰·design.md·WCAG AA 검증)는 머지. 2단계(딥바이올렛 사이드바·슬랙 채널형 채팅)는 PR #19로 열려 있음.
KAN-22	UI 트렌디 리스타일링	#18 오픈	리뷰 중	인디고 테마·next-themes 다크 토크·sticky 헤더. KAN-21 2단계와 같은 파일 3개를 서로 다르게 고친다 → 5장 결정 항목.

기능 개수보다 중요한 건, 22개 이슈를 지나며 **재사용 가능한 원칙**이 남았다는 점이다. 아래 8가지는 앞으로 추가할 모든 기능(채널·스레드·문서 트리)이 그대로 따라야 할 규약이다.

01 멀티테넌시 격리는 쿼리 수준에서

테넌트 데이터는 예외 없이 `where: { orgId }` 로 스코프한다. 단건 update/delete API 대신 `*Many` 를 써서, 남의 워크스페이스 id를 알라도 애초에 매칭되지 않게 한다.

→ 신규 채널·스레드·문서 트리도 동일 규약을 상속한다.

02 에러 계약 (ActionResult)

Server Action은 판별 유니언만 반환하고, DB 예외는 guarded 래퍼가 흡수한다. 클라이언트가 불투명한 digest 에러를 받는 경로를 없었다.

→ 화면은 항상 사람이 읽을 수 있는 한국어 메시지를 받는다.

03 웹훅은 역등 + 단일 트랜잭션

멤버십 이벤트에 임베드된 org-user-membership을 한 트랜잭션에 upsert한다. 웹훅이 어떤 순서로 도착해도 FK가 깨지지 않는다.

→ 외부 시스템 동기화의 표준 패턴을 이미 갖췄다.

04 순서 역전 가드 (tombstone + updatedAt)

재시도로 지연 도착한 upsert가 삭제된 행을 부활시키지 못하게 tombstone을 영구 보관하고, 원본 `updated_at` 보다 오래된 이벤트는 무시한다.

→ 미래를 위한 판단에 쓰게 될 때 보안 사고로 번지는 경로를 미리 차단.

05 authz의 근거는 세션, 미러가 아니다

역할은 Clerk 세션 클레임으로만 판단한다. DB 미러의 `role` 은 웹훅 지연으로 stale할 수 있어 표시용으로만 쓴다.

→ "보이는 값"과 "판단하는 값"을 분리한 것이 이 프로젝트의 핵심 authz 원칙.

06 테넌트 수명 정책

조직 삭제 = 그 조직 데이터 파기(Cascade). Restrict는 웹훅을 500 루프에 빠뜨리고 SetNull은 스코프를 잃기 때문에 Cascade만 정합적이라는 결론.

→ 단, ChatMessage는 FK가 없어 아직 이 정책 밖에 있다(KAN-19).

07 낙관적 UI는 타깃 롤백으로

실패 시 전체 스냅샷을 되돌리면 그 사이의 다른 변이를 덮어쓴다. 실패한 항목만 함수형으로 되돌린다.

→ 실시간(채팅·보드) 기능에서 특히 중요한 규칙.

08 PR마다 자체 리뷰를 프로세스로

1인 개발의 사각지대를 메우기 위해 PR마다 다중 관점 리뷰 → 발견별 독립 검증 → 확정분만 반영, 설계 변경이 필요한 건 후속 티켓으로 분리한다.

→ KAN-5-6-7-9-12-19가 모두 이 방식으로 태어난 티켓이다.

기술 스택 현재형

Next.js 16(App Router·Turbopack) + React 19 · TypeScript strict · PostgreSQL 17 + Prisma 7 · Clerk(Organization=워크스페이스) · **shadcn/ui** + **Tailwind v4**(KAN-20에서 Astryx/StyleX 대체) · Tiptap 3 · Pusher · dnd-kit · zod · pnpm 10 / Node 22.
기획서 v1.0의 "Astryx/StyleX 단독" 기술 표는 이 전환으로 무효 — 문서 갱신이 필요하다(NEW-06).

결정이 필요한 1건 — UI 리스타일 PR 2개가 서로를 덮어쓴다

PR #18(KAN-22)과 PR #19(KAN-21 2단계)가 같은 베이스 브랜치에 동시에 열려 있고,

`globals.css` · `layout.tsx` · `NavLink.tsx` 3개 파일을 서로 다른 방향으로 고친다 — 한쪽은 인디고 테마 + 상단 헤더 내비를 추가하고, 다른 쪽은 바이올렛 테마 + 슬랙형 좌측 사이드바를 넣으며 그 헤더 내비를 삭제한다. 먼저 머지되는 쪽이 다른 쪽을 충돌로 밀어낸다.

권장 — 시간순·의도상 PR #19(슬랙형 앱 셸)가 "변화가 크게 안 보인다"는 피드백에 대한 최종 답이므로 **#19를 채택**하고, #18에서 겹치지 않는 폴리시(다크 토글 방식, 카드 hover·포커스 링, EmptyState 아이콘)만 흡수한 뒤 #18은 닫는다. 그 후 KAN-20 브랜치를 main으로 올려 Phase 5를 종료한다.

항목	근거	성격	위험 · 권장 조치
UI 리스타일 트랙 2개 병존	PR #18 · #19 충돌 파일 3	즉시	Phase 5 산출물이 통째로 main에 안 올라간 상태로 정지. 위 권장안대로 단일화 → main 머지. (NEW-01)
ChatMessage 수명·쓰기 가드 부재	<code>services/chat.ts</code> KAN-19	보안	조직 삭제 후에도 메시지가 남고, stale 세션이 삭제된 org로 계속 쓸 수 있다. 정리 경로가 없어 영구 축적. (NEW-02)
자동 테스트 0건	테스트 파일 없음	구조	멀티테넌시 격리·권한 규칙처럼 깨지면 보안 사고 인 로직이 회귀 감지 없이 방치. 서비스 계층 단위 테스트부터. (NEW-03)
CI 파이프라인 없음	<code>.github/workflows</code> 없음	구조	tsc-lint-build가 로컬 수동 확인에만 의존. PR 필수 체크로 승격. (NEW-04)
배포 환경 없음 · 웹훅 미검증	로컬 docker만	기반	Clerk 웹훅은 localhost로 직접 수신이 불가해 실환경 e2e가 한 번도 안 돌았다 . 스테이징 배포로 해소. (NEW-05)
Jira 상태·제목이 실제와 불일치	KAN-12·16·17·18·20~22	위생	완료·머지된 이슈 다수가 "해야 할 일"로 남아 번다운·보드가 무의미. KAN-20은 제목 자체가 옛 범위. 샘플 3건 정리. (NEW-06)
기획서 v1.0 stale	docs/기획서.pdf	위생	기술 표가 Astryx/StyleX 기준이고 로드맵이 Phase 5에서 끝난다. 본 리포트 기준으로 v2 갱신. (NEW-06)
tombstone 가드 코드 중복	<code>services/notes.ts</code>	정리	보드에서 <code>assertNotTombstoned</code> 로 공유 추출했으나 notes는 인라인 중복이 남아 있다. NEW-02와 함께 DRY.
메시지·노트 전량 로드	<code>chat.findMany</code>	성능	페이지네이션이 없어 데이터가 쌓이면 초기 로드가 선형으로 악화. (NEW-08)

목표는 "노선처럼 쓰는 문서 + 슬랙처럼 쓰는 대화"가 **하나의 워크스페이스**에서 도는 제품이다. 현재와 목표 사이의 격차를 기능 단위로 분해하면 아래 5단계가 된다. 순서의 논리는 하나다 — **고칠 수 없는 것(테스트·CI·배포)을 먼저 세우고, 그 위에 사용자가 보는 기능을 쌓는다.**

단계	목표	티켓	우선	왜 이 순서인가
Phase 6 정리 · 기반	열린 PR 단일화, 테스트·CI·스테이징 확보	6건	P0	지금 기능을 더 쌓으면 검증되지 않은 코드가 같이 늘어난다. 여기서 안전망을 먼저 만든다.
Phase 7 슬랙형 채팅	채널·스레드·리액션·멘션·읽음·첨부	9건	P1	현재 완성도가 가장 낮고(25%), 앱의 기본 착지점이 채팅이라 체감 효과가 가장 크다.
Phase 8 노선형 문서	문서 트리·블록 확장·공동 편집·코멘트	6건	P1	실시간 공동 편집은 이 프로젝트의 기술적 하이라이트 — 채팅 실시간 경험을 쌓은 뒤 도전한다.
Phase 9 플랫폼 통합	통합 검색·보드 고도화·초대/온보딩·알림	4건	P2	기능이 세 개 다 자란 뒤에야 "가로지르는" 기능(검색·알림)이 의미를 갖는다.
Phase 10 운영 품질	E2E·레이트 리밋·관측성·성능/접근성	4건	P2	포트폴리오로 보여줄 때 "돌아간다"가 아니라 "운영된다"를 증명하는 층.

제품 관점의 격차 요약

슬랙이라 부르려면 필요한 것

- 채널(공개/비공개) — 지금은 워크스페이스당 방 1개
- 스레드 답글 · 이모지 리액션
- 멘션과 알림 · 읽음/안읽음
- 파일·이미지 첨부 · DM

노선이라 부르려면 필요한 것

- 문서 트리(중첩 페이지) · 사이드바 내비
- 슬래시 커맨드 · 체크리스트 · 이미지 · 테이블
- 실시간 공동 편집(다중 커서)
- 인라인 코멘트 · 버전 히스토리 · 공유 링크

● 완료 · main 반영됨

16

KAN-4
노트 CRUD API
멀티테넌시 골격

PR #1

KAN-5
fresh clone 빌드 복구

PR #2

KAN-6
에러 계약 + 원자성

PR #3

KAN-7
PR #1 리뷰 cleanup

PR #9

KAN-8
노트 UI

PR #4

KAN-9
낙관적 업데이트

PR #7

KAN-10
Clerk 인증 뼈대
Org=워크스페이스

PR #5

KAN-11
웹훅 → DB 동기화

PR #6

KAN-12
순서 역전 가드
Jira 상태 정정 필요

PR #12

KAN-13
멤버 목록 UI

PR #8

KAN-14
Note FK + 작성자

PR #10

KAN-15
실시간 채팅 MVP
Pusher

PR #11

KAN-16
Tiptap 에디터

PR #13

KAN-17
칸반 보드 MVP
dnd-kit

PR #14

KAN-18
역할 기반 권한

PR #15

KAN-20
UI 스택 전환
shadcn + Tailwind v4

PR #16

● 리뷰 중

2

KAN-21
디자인 시스템 + 슬랙형 앱 셀
1단계 머지(#17) · 2단계 오픈

PR #19

KAN-22
트렌디 리스타일링
KAN-21과 파일 충돌 → 단일화 결정 필요

PR #18

● 대기

1

KAN-19
ChatMessage 테넌트 수명 정렬
org 삭제 시 파기 + stale 쓰기 차단

보안

● 정리 대상

3

KAN-1
협업 SaaS 프로젝트

KAN-2
작업 2

KAN-3
하위 작업 2.1
Jira 온보딩 잔재

보드가 말해주는 것

- 완료 16장은 모두 main에 있다. 그런데 그 위에 쌓인 KAN-21-22는 아직 아니다. 디자인 시스템 앱 셀 다크 모드 — 가장 눈에 보이는 작업 세 덩이가 브랜치에 묶여 있고, "완료"와 "출시"가 벌어진 이 간극이 길어질수록 충돌 비용이 커진다.
- 대기 열이 1장뿐인 건 좋은 신호가 아니다. 백로그가 비어 있다는 뜻이 아니라, 다음에 할 일이 티켓으로 적히지 않았다는 뜻이다 — 그래서 이 리포트 8장에서 29건을 채운다.
- 정리 대상 3장은 Jira 온보딩 샘플이다. 하나가 "진행 중" 상태라 보드 통계를 계속 오염시킨다.

완료 16건의 구성

기능 티켓 11건 · 자체 리뷰가 만들어진 후속 티켓 5건(KAN-5-6-7-9-12). 여기에 아직 열려 있는 KAN-19까지 더하면 리뷰 파생 티켓은 6건 — 전체의 3분의 1 가깝이가 "PR을 넘기기 전에 스스로 찾아낸 문제"에서 나왔다는 뜻이고, 이 프로젝트가 굴러온 방식을 그대로 보여준다.

각 카드는 그대로 Jira 티켓이 된다. **P0**=다음 스프린트, **P1**=핵심 제품 기능, **P2**=완성도. 규모는 **S**(반나절~1일) · **M**(2~3일) · **L**(1주+) 추정이다.

Phase 6 · 정리와 기반

기능을 더 쌓기 전에 세울
안전망. 여기부터
시작한다.

NEW-01 **P0 S**
UI 리스타일 트랙 단일화
PR #18/#19 충돌 해소 → KAN-20
브랜치 main 머지, Phase 5 종료

NEW-02 **P0 S**
ChatMessage 수명 정렬 (**KAN-19**)
org 삭제 시 메시지 파기 +
tombstone 쓰기 가드 + notes DRY

NEW-03 **P0 M**
테스트 하네스 도입
Vitest + 서비스 계층. 멀티테넌트·
권한 회귀 테스트를 1순위로

NEW-04 **P0 S**
CI 파이프라인
GitHub Actions — tsc-lint-build-test를
PR 필수 체크로

NEW-05 **P0 M**
스테이징 배포 + 웹훅 실연동
Vercel + 관리형 Postgres. Clerk 웹훅
e2e를 처음으로 실제 검증

NEW-06 **P1 S**
문서·티켓 정합성 회복
Jira 상태/제목 정정, 샘플 3건 정리,
기획서 v2 갱신

Phase 7 · 슬랙형 채팅

완성도 25% → 목표 80%.
체감 변화가 가장 큰 구간.

NEW-07 **P1 L**
채널 모델 도입
워크스페이스당 다중 채널(공개/
비공개), 채널 CRUD·참여. Pusher
채널을 org→채널 단위로

NEW-08 **P1 M**
메시지 히스토리 페이지네이션
커서 기반 위로 무한 스크롤 — 현재
전량 로드 구조 해소

NEW-09 **P1 M**
스레드(답글)
parentId + 스레드 패널, 답글 수 표시

NEW-10 **P1 S**
이모지 리액션
(메시지, 유저, 이모지) 유니크 · 실시간
반영

NEW-11 **P1 L**
멘션 + 인앱 알림 센터
@user/@channel 파싱, Notification
모델, 실시간 뱃지

NEW-12 **P1 M**
읽음 상태 · 안읽음 뱃지
멤버×채널 lastReadAt, 사이드바
미읽음 카운트

NEW-13 **P2 S**
프레즌스 · 타이핑 인디케이터
Pusher presence 채널 활용

NEW-14 **P1 M**
파일 · 이미지 첨부
오브젝트 스토리지 + 서명 URL, org
스코프 접근 제어

NEW-15 **P2 M**
DM (1:1 · 그룹)
채널 모델 확장 — 멤버십 기반 비공개
채널

Phase 8 · 노선형 문서

평면 노트 → 중첩 문서
워크스페이스.

NEW-16 **P1 M**
문서 트리 + 사이드바 내비
Note.parentId 중첩, 드래그 재배치,
즐거찾기

NEW-17 **P1 M**
에디터 블록 확장
슬래시 커맨드·체크리스트·이미지·
테이블. **zod** 화이트리스트 동반 확장
필수

NEW-18 **P1 L**
실시간 공동 편집
Yjs CRDT + 서버 퍼시스턴스, 다중
커서. 기술적 하이라이트

NEW-19 **P2 M**
인라인 코멘트
텍스트 범위 앵커 + 스레드, 멘션 연동

NEW-20 **P2 M**
버전 히스토리 · 복원
스냅샷 저장, diff 뷰

NEW-21 **P2 S**
공개 공유 링크
읽기 전용 퍼블리시 — 테넌트 밖
접근의 첫 사례라 authz 설계 주의

Phase 9 · 플랫폼 통합

세 기능을 가로지르는 층.

NEW-22

P1M

통합 검색

Postgres 전문검색으로 노트·메시지·카드를 한 번에 (org 스코프 필수)

NEW-23

P1M

보드 고도화

카드 상세(담당자·라벨·마감일·설명) + 노트 연결

NEW-24

P1M

멤버 초대 · 역할 관리 UI + 온보딩

F1의 마지막 조각 — 조직 생성/초대 플로우

NEW-25

P2M

알림 설정 + 이메일 다이제스트

채널별 알림 수준, 미읽음 요약 메일

Phase 10 · 운영 품질

"돌아간다"에서 "운영된다"로.

NEW-26

P2M

E2E 테스트 (Playwright)

로그인→조직→노트/채팅/보드 핵심 플로우

NEW-27

P2S

레이트 리밋 · 남용 방어

Server Action·웹훅 엔드포인트 보호

NEW-28

P2S

관측성 — 에러 모니터링·구조적 로깅

웹훅 실패·액션 예외 추적

NEW-29

P2M

성능 · 접근성 마감

N+1 제거, 모바일 반응형, a11y 감사

합계 29장 = Phase 6 (6) + Phase 7 (9) + Phase 8 (6) + Phase 9 (4) + Phase 10 (4). 이 중 **NEW-02**는 기존 **KAN-19**를 승계하므로 Jira에 새로 만들 티켓은 **28건**이다.

달력 날짜가 아니라 상대적 소요로 그린다(1인 개발, 학습 병행). 각 마일스톤은 "여기까지 하면 남에게 이렇게 소개할 수 있다"는 기준으로 끊었다.

마일스톤	1주차	2~3주	4~6주	7~9주	10주+
M1 기반 정비 Phase 6	P0 6건				
M2 "슬랙이라 부를 수 있다" Phase 7		채널·스레드·리액션·멘션·읽음·첨부			
M3 "노션이라 부를 수 있다" Phase 8				트리·블록·공동 편집	
M4 통합 제품 Phase 9					검색·보드·초대·알림
M5 운영 가능 Phase 10					E2E·관측·성능

지금 당장 할 세 가지

순서	할 일	티켓	완료 판정
1	UI 트랙 단일화 후 main 머지	NEW-01 KAN-21-22	PR 하나로 정리되어 main에 올라가고, 열린 PR이 0이 된다. Phase 5가 실제로 끝난다.
2	KAN-19 처리 (조직 삭제 시 메시지 파기 + stale 세션 쓰기 차단)	NEW-02	웹훅 e2e로 org 삭제 → 메시지 0건 확인, 삭제 후 전송 시도 거부 확인.
3	테스트 + CI 세우기 (멀티테넌시·권한 회귀부터)	NEW-03-04	남의 org 데이터 접근 시도가 테스트로 막히고, 그 테스트가 PR마다 자동 실행된다.

이 순서를 원하는 이유

지금까지 22개 이슈의 3분의 1이 "리뷰가 찾아낸 문제"에서 나왔다. 사람이 매번 찾아낸 것이고, 그래서 지금까지는 통했다. 하지만 채널·스레드·공동 편집이 붙으면 org 스코프를 빠뜨린 쿼리 하나가 곧 데이터 유출이 된다. 기능을 늘리기 전에 그 규칙을 기계가 지키게 만드는 것이 M1의 목적이다.

현재 스키마 (8모델 · 마이그레이션 6회)

모델	도입	역할과 제약
Note	KAN-4 → 14 → 16	문서. <code>orgId</code> FK(Cascade) · <code>authorId</code> FK(SetNull) · <code>content</code> 는 Tiptap doc JSON 직렬화 문자열.
User	KAN-11 → 12	Clerk 사용자 미러. <code>clerkUpdatedAt</code> 으로 순서 역전 가드.
Organization	KAN-11 → 12	Clerk 조직(=워크스페이스) 미러. 삭제 시 소속 데이터 Cascade 파기.
Membership	KAN-11 → 12	사용자↔조직 소속과 role. role은 표시용 — 권한 판단은 세션 클레임.
ClerkTombstone	KAN-12	삭제된 Clerk id의 영구 기록. 지연 이벤트의 부활 차단.
ChatMessage	KAN-15	워크스페이스 채팅(방 1개). FK 없음 — 웹훅 순서 경합 회피 목적이나 그 대가로 KAN-19 부채가 생겼다.
BoardColumn	KAN-17	칸반 컬럼. <code>orgId</code> Cascade, 정수 position.
BoardCard	KAN-17	칸반 카드. <code>columnId</code> Cascade + <code>authorId</code> SetNull, 이동 시 대상 컬럼 position 재작성.

앞으로 추가될 모델 (계획 기준)

Channel · ChannelMembership(읽음 상태 포함) · MessageReaction · Notification · Attachment · Comment · NoteVersion · ShareLink — 모두 기존 규약(`orgId` 스코프 + Cascade 수명)을 상속한다. ChatMessage는 채널 도입(NEW-07) 시 `channelId` FK를 갖게 되므로, **KAN-19의 수명 문제도 이때 구조적으로 해소된다.**

이 문서를 만든 근거

Jira REST(프로젝트 KAN 전체 22건) · git 로그 61커밋 · GitHub PR #1~#19 · `prisma/schema.prisma` 와 `src/` 소스 직접 확인 · 기획서 v1.0(2026-07-23). 상태 표기는 Jira 필드가 아니라 **코드와 PR의 실제 상태**를 우선했다.